Program Name and Semester: BCA 4TH Semester
Course Name (Course Code): FULL STACK DEVELOPMENT -1(BCA47111(P))

Class: BCA2023-4TH SEM
Academic Session: 2025-26

Department of Computational Sciences

Laboratory Manual

(BCA2024, SEMESTER 4TH,

FULL STACK DEVELOPMENT-1 and BCA47111(P))

Table of Contents

Experiment/Lab Exercise/Activity No.	Name of the Experiment/Activity/Exercise	Page Number(s)
1	Create a Node.js script demonstrating variables, functions, conditionals, loops, objects, arrays, and asynchronous code.	6-8
2	a) Write a Node.js program to take user input, add two numbers, and display the result. b) b. Create a Node.js script to display system environment variables.	8-10
3	a) Write a Node.js program to demonstrate command-line arguments and parse their values. b) b. Explain REPL in Node.js with examples of real-time JavaScript testing.	10-13
4	Write a Node.js script for the REPL environment, including functions, array methods, and asynchronous operations.	13-15
5	Write a Node.js program to calculate the execution time of multiple functions.	15-17
6	Write a Node.js program to explain asynchronous and synchronous functions with examples.	17-19
7	a) Write a Node.js program to describe the core `path` module. b) Write a Node.js program to describe the core modules `path` and `fs`.	19-21
8	a) Create a Node.js program demonstrating the use of Buffers, including: Creating buffers with `Buffer.from()` and `Buffer.alloc()`. Writing data using `write()`. Slicing buffers using `slice()`.	21-23
9	a) Write a Node.js program to describe local modules with two functions. b) Write a Node.js program to describe local modules with an object.	23-26
10	Create a `package.json` file for a Node.js project and describe its setup with an external package like `dotenv`.	26-28

Name of the Faculty: SAMRAT KUNDU
Designation and Department: Assistant Professor (Computational Sciences)
Brainware University, Kolkata

11	a) Write a Node.js program to create a server using the `http` core module. b) Extend the server to handle routes and process different HTTP methods (`GET`, `POST`, `PUT`, `DELETE`).	28-32
12	Write a Node.js program to perform CRUD operations (Create, Read, Update, Delete) on a file using the `fs` core module.	32-34
13	a) Create a Node.js project named "Calculator" with a server handling routes for: b) A home page (`/`) displaying a welcome message and a link to the calculator page. c) A calculator page (`/calculator`) with a form to input two numbers and calculate their sum. d) A result page (`/calculate-result`) parsing user input, calculating the sum using a separate addition module, and displaying the result.	34-37
14	a) Create a Node.js project using Express that: b) Responds with both HTML and JSON. c) Serves different HTML files at different routes.	37-40
15	a) Extend the Express project to: b) Serve different HTML files at different routes and perform a calculation (e.g., add two numbers from a user form). c) Implement routing with dynamic parameters in the URL. d) Use application-level middleware for tasks like logging or handling errors.	40-44

Experiment No 1

1. **Aim/Purpose of the Experiment:**

ASSIGNMENT 1: To demonstrate the fundamental programming concepts in Node.js, including variables, functions, conditionals, loops, objects, arrays, and asynchronous code execution, helping students understand the core building blocks of Node.js programming.

ASSIGNMENT 2: To write a Node.js program that takes user input, performs arithmetic operations (addition), and displays the result, allowing students to understand input handling and basic calculation tasks in Node.js.

ASSIGNMENT 3: To demonstrate how to work with command-line arguments in Node.js by writing a program to parse and process values, providing students with an understanding of how to handle external inputs in a Node.js application.

ASSIGNMENT 4: To explore and utilize the Node.js REPL environment by writing scripts that incorporate functions, array methods, and asynchronous operations, giving students a hands-on approach to testing JavaScript code in real-time.

ASSIGNMENT 5: To create a Node.js program that calculates and displays the execution time of multiple functions, providing insights into performance measurement and benchmarking within the Node.js environment.

ASSIGNMENT 6: To explain and demonstrate the difference between asynchronous and synchronous functions in Node.js, showing how each type handles tasks like file I/O, HTTP requests, and timers.

ASSIGNMENT 7: To write a Node.js program that uses the core path module for working with file paths, helping students understand how to handle and manipulate file system paths efficiently.

ASSIGNMENT 8: To explore Buffers in Node.js by creating, writing, and slicing buffers using the Buffer.from() and Buffer.alloc() methods, helping students understand binary data manipulation in memory.

ASSIGNMENT 9: To write a Node.js program that creates local modules, either with functions or objects, demonstrating how to organize and reuse code in modular Node.js applications.

ASSIGNMENT 10: To create a package.json file for a Node.js project and integrate an external package like dotenv, teaching students how to manage project dependencies and environment variables in a Node.js environment.

ASSIGNMENT 11: To build a basic web server using Node.js's core http module, and extend the server to handle various HTTP methods (GET, POST, PUT, DELETE) and routing, giving students a practical understanding of web server creation.

ASSIGNMENT 12: To implement CRUD operations (Create, Read, Update, Delete) on a file using the fs core module, providing students with hands-on experience in file handling and data management in Node.js.

ASSIGNMENT 13: To create a Node.js web project named "Calculator" that serves multiple pages with routing, including a homepage, calculator page, and a result page, demonstrating the full-stack development process with Node.js and routing.

ASSIGNMENT 14: To develop a Node.js project using Express that serves HTML and JSON responses, giving students an understanding of building web applications that handle different types of content and serve dynamic routes.

ASSIGNMENT 15: To extend an Express project by implementing dynamic routing with URL parameters and application-level middleware for tasks like logging and error handling, equipping students with the skills to build advanced Node.js applications with improved functionality.

2. Learning Outcomes

ASSIGNMENT 1:
Students will gain a strong understanding of basic programming constructs in Node.js, including variables, functions, loops, conditionals, objects, arrays, and asynchronous code execution, enabling them to write efficient Node.js scripts for various applications.

ASSIGNMENT 2:
Students will learn how to handle user input in Node.js, perform basic arithmetic operations, and display the output, thereby gaining practical skills in interactive Node.js applications.

ASSIGNMENT 3:
Students will develop the ability to handle and parse command-line arguments in Node.js, enhancing their skills in creating interactive programs that take user input from the command line.

ASSIGNMENT 4:
Students will become proficient in using the REPL environment for real-time JavaScript testing, exploring functions, arrays, and asynchronous operations to experiment and debug code efficiently.

ASSIGNMENT 5:
Students will learn how to measure the performance of their Node.js applications by calculating the execution time of multiple functions, gaining insights into performance optimization and benchmarking techniques.

ASSIGNMENT 6:
Students will distinguish between asynchronous and synchronous functions in Node.js, understanding their use cases and behavior in handling I/O operations and concurrent tasks.

ASSIGNMENT 7:
Students will gain expertise in using Node.js's core path module to handle and manipulate file paths, ensuring that their applications work efficiently with file system paths.

ASSIGNMENT 8:
Students will understand the concept of Buffers in Node.js, learning to manipulate binary data effectively using Buffer methods like Buffer.from(), Buffer.alloc(), and write(), which is crucial for dealing with binary data in applications.

ASSIGNMENT 9:
Students will gain the knowledge to create and work with local modules in Node.js, allowing them to structure their applications efficiently by organizing reusable code into modules.

ASSIGNMENT 10:
Students will learn how to configure and manage Node.js project dependencies using package.json and external packages like dotenv, allowing for environment variable management and effective dependency handling.

ASSIGNMENT 11:
Students will acquire the skills to create a basic web server using Node.js's http module and extend it to handle various HTTP methods (GET, POST, PUT, DELETE), giving them a foundational understanding of web server development.

ASSIGNMENT 12:
Students will learn to perform file-based CRUD operations using Node.js's fs module, equipping them with the tools to manage and manipulate files within their applications.

ASSIGNMENT 13:
Students will develop a practical understanding of full-stack web development by building a simple calculator application in Node.js, including server-side handling of routes and user input.

ASSIGNMENT 14:
Students will gain hands-on experience in building dynamic web applications with Express that serve both HTML and JSON responses, learning to manage different types of content in web applications.

ASSIGNMENT 15:
Students will enhance their Express development skills by implementing dynamic routing, URL parameters, and middleware functionalities (such as logging and error handling), improving their ability to build scalable and maintainable applications.

3. Prerequisites

ASSIGNMENT 1:
Basic understanding of JavaScript programming, including variables, functions, loops, and objects.

ASSIGNMENT 2:
Knowledge of basic JavaScript operations, such as input handling and arithmetic operations.

ASSIGNMENT 3:
Familiarity with JavaScript command-line applications and basic Node.js environment setup.

ASSIGNMENT 4:
Understanding of JavaScript functions and arrays, and familiarity with Node.js REPL environment.

ASSIGNMENT 5:
Basic understanding of JavaScript functions and performance measurement concepts.

ASSIGNMENT 6:
Knowledge of JavaScript asynchronous programming (callbacks, promises, and async/await).

ASSIGNMENT 7:
Familiarity with file system operations and Node.js's core modules like path.

ASSIGNMENT 8:
Understanding of binary data handling and familiarity with Node.js Buffer API.

ASSIGNMENT 9:
Basic understanding of Node.js modules and the concept of modular programming.

ASSIGNMENT 10:
Familiarity with Node.js project structure and understanding the use of package.json for dependency management.

ASSIGNMENT 11:
Basic knowledge of HTTP and JavaScript, and an understanding of how web servers work.

ASSIGNMENT 12:
Knowledge of file I/O operations in JavaScript and understanding the Node.js fs module.

ASSIGNMENT 13:
Basic knowledge of Node.js, HTML, and web server concepts for building dynamic applications.

ASSIGNMENT 14:
Familiarity with Express.js and web application development with routing and dynamic responses.

ASSIGNMENT 15:
Basic understanding of Express.js, middleware concepts, and routing in web applications.

4. Materials/Equipment/Apparatus / Devices/Software required

Software Required:

- **Node.js**
- **Text Editor** (VS Code, Sublime Text, or any preferred code editor)
- **Command-line terminal**
- **Web browser** (for testing HTTP requests and server responses)
- **`dotenv` package** (via npm for environment management)
- **File system access** (local or virtual for file operations)

Hardware Required:

- **Computer** with internet access for installing Node.js and running scripts
- **Basic hardware setup** to support development and testing of Node.js applications

5. Introduction and Theory:

ASSIGNMENT 1:

Introduction

Node.js is a runtime environment that allows you to run JavaScript code outside the browser. This script covers essential programming constructs using Node.js:

1. **Variables**: Declaration using `let`, `const`, and `var`.
2. **Functions**: Both synchronous and asynchronous.
3. **Conditionals**: Using `if-else` and `switch`.
4. **Loops**: Using `for`, `while`, and `forEach`.
5. **Objects and Arrays**: Demonstrates CRUD operations.
6. **Asynchronous Code**: Using `setTimeout` and `Promises`.

Solution

```
// 1. Variables
let message = "Welcome to Node.js!";
const appName = "NodeJS Demo";
var count = 5; // Old-style declaration

console.log(`${appName}: ${message}`);
```

```
// 2. Functions
function greetUser(name) {
    return `Hello, ${name}!`;
}
console.log(greetUser("Samrat"));

// 3. Conditionals
const checkEvenOdd = (num) => {
    if (num % 2 === 0) {
        console.log(`${num} is even.`);
    } else {
        console.log(`${num} is odd.`);
    }
};
checkEvenOdd(7);

// 4. Loops
let numbers = [1, 2, 3, 4, 5];
console.log("Using a 'for' loop:");
for (let i = 0; i < numbers.length; i++) {
    console.log(numbers[i]);
}

console.log("Using a 'while' loop:");
let i = 0;
while (i < numbers.length) {
    console.log(numbers[i]);
    i++;
}

console.log("Using 'forEach':");
numbers.forEach((num) => console.log(num));

// 5. Objects and Arrays
let user = {
    name: "Samrat",
    age: 25,
    hobbies: ["reading", "coding", "gaming"],
};

// Accessing object properties
console.log(`Name: ${user.name}`);
console.log(`Age: ${user.age}`);
console.log("Hobbies:", user.hobbies);

// Updating object properties
user.age = 26;
user.hobbies.push("music");
console.log("Updated User:", user);

// 6. Asynchronous Code
const asyncExample = () => {
    console.log("Asynchronous operation starts...");

    setTimeout(() => {
        console.log("This message is delayed by 2 seconds.");
```

```
    }, 2000);

    console.log("Asynchronous operation ends...");
};
asyncExample();

// Promises
const fetchData = () => {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            resolve("Data fetched successfully!");
        }, 3000);
    });
};

fetchData()
    .then((data) => console.log(data))
    .catch((err) => console.error(err));
```

Key Points

1. **Variables**: Use `let` for block-scoped variables, and `const` for constants.
2. **Functions**: Define reusable code blocks.
3. **Conditionals**: Use to control the flow of execution based on conditions.
4. **Loops**: Iterate over arrays and perform repetitive tasks.
5. **Objects and Arrays**: Store and manipulate collections of data.
6. **Asynchronous Code**: Manage non-blocking operations using `setTimeout`, Promises, or async/await.

ASSIGNMENT 2:

---

a) Node.js Program to Take User Input, Add Two Numbers, and Display the Result

This script includes validation, error handling, and a confirmation of the input provided.

```
// Import the readline module
const readline = require('readline');

// Create an interface for reading input from the user
const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
});

// Function to validate if the input is a valid number
const isValidNumber = (input) => {
    const num = parseFloat(input);
    return !isNaN(num);
```

```
};

// Prompt the user for two numbers
console.log("This program adds two numbers provided by the user.");

rl.question('Enter the first number: ', (num1) => {
    if (!isValidNumber(num1)) {
        console.log("Error: The input is not a valid number.");
        rl.close();
        return;
    }

    rl.question('Enter the second number: ', (num2) => {
        if (!isValidNumber(num2)) {
            console.log("Error: The input is not a valid number.");
            rl.close();
            return;
        }

        // Convert inputs to numbers and calculate the sum
        const sum = parseFloat(num1) + parseFloat(num2);

        // Display the result
        console.log(`You entered: ${num1} and ${num2}`);
        console.log(`The sum of ${num1} and ${num2} is: ${sum}`);

        // Close the readline interface
        rl.close();
    });
});
```

Features Added:

1. Input validation to ensure only numbers are processed.
2. Clear prompts and error messages for invalid inputs.
3. Echoes the inputs back to the user for confirmation.

---

b) Node.js Script to Display System Environment Variables

This script provides a detailed, formatted display of environment variables and allows users to query a specific variable.

```
// Display all system environment variables in a readable format
console.log("Displaying All System Environment Variables:\n");

// Loop through each environment variable and display it
Object.entries(process.env).forEach(([key, value]) => {
    console.log(`${key}: ${value}`);
});

// Optional: Allow the user to query specific environment variables
```

```
const readline = require('readline');
const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
});

rl.question("\nDo you want to query a specific environment variable? (yes/no): ",
(response) => {
    if (response.toLowerCase() === "yes") {
        rl.question("Enter the name of the environment variable: ", (variable) => {
            if (process.env[variable]) {
                console.log(`\nValue of ${variable}: ${process.env[variable]}`);
            } else {
                console.log(`\nEnvironment variable '${variable}' not found.`);
            }
            rl.close();
        });
    } else {
        console.log("\nExiting the script.");
        rl.close();
    }
});
```

Features Added:

1. Displays all environment variables with clear formatting.
2. Allows users to query the value of a specific environment variable interactively.
3. Handles cases where the specified environment variable does not exist.

How to Run the Scripts

1. **Save the Scripts:**
   - Save the first script as `addNumbersDetailed.js`.
   - Save the second script as `envVariablesDetailed.js`.
2. **Run the Scripts:**
   - Open a terminal and navigate to the folder containing the files.
   - Run the addition program: `node addNumbersDetailed.js`.
   - Run the environment variables script: `node envVariablesDetailed.js`.
3. **Interact with the Scripts:**
   - Follow the prompts in the addition program to input numbers and get the result.
   - Explore environment variables in the second script and optionally query specific variables.

ASSIGNMENT 3:

---

a) Node.js Program to Demonstrate Command-Line Arguments and Parse Their Values

Command-line arguments in Node.js can be accessed using `process.argv`. Here's a detailed example:

```
// Access command-line arguments using process.argv
const args = process.argv.slice(2); // Slice to exclude 'node' and the script name

// Function to parse arguments into key-value pairs
function parseArguments(args) {
    const parsedArgs = {};
    args.forEach((arg) => {
        const [key, value] = arg.split("=");
        if (key && value) {
            parsedArgs[key] = value;
        }
    });
    return parsedArgs;
}

// Check if arguments were provided
if (args.length === 0) {
    console.log("No arguments provided. Please pass arguments in the format
key=value.");
    console.log("Example: node script.js name=Samrat age=25");
} else {
    // Parse and display the arguments
    const parsedArgs = parseArguments(args);
    console.log("Parsed Command-Line Arguments:");
    console.table(parsedArgs); // Display the arguments in table format
}
```

How to Run:

1. Save the script as `parseArgs.js`.
2. Run it in the terminal with arguments:
3. `node parseArgs.js name=Samrat age=25 location=India`

Expected Output:

```
Parsed Command-Line Arguments:
```

(index)	Values
name	'Samrat'
age	'25'
location	'India'

---

b) Explain REPL in Node.js with Examples of Real-Time JavaScript Testing

REPL (Read-Eval-Print Loop):

- **Read**: Reads user input (a JavaScript expression).

- **Eval**: Evaluates the input.
- **Print**: Prints the result.
- **Loop**: Waits for the next input.

REPL is an interactive environment that allows real-time JavaScript testing in Node.js. It is particularly useful for quickly experimenting with code.

Starting REPL

1. Open a terminal.
2. Type `node` and press Enter to start the REPL environment.

Key Features and Examples

Basic Arithmetic:

```
> 2 + 3
5
> 10 * 5
50
```

Variable Declarations:

```
> let name = "Samrat";
undefined
> name
'Samrat'
```

Function Definitions and Invocations:

```
> function greet(name) { return `Hello, ${name}!`; }
undefined
> greet("Samrat")
'Hello, Samrat!'
```

Accessing Built-in Modules:

```
> const os = require('os');
undefined
> os.platform()
'linux' // or your OS platform
```

Multiline Input: Use `.editor` for multiline input:

```
> .editor
// Entering editor mode (Ctrl+D to exit)
const add = (a, b) => a + b;
add(5, 3);
(Ctrl+D)
8
```

Loading External Files: You can load and test scripts directly in REPL:

```
> .load yourScript.js
```

Exiting REPL

Type `.exit` or press `Ctrl+C` twice.

Real-Time Use Cases of REPL

1. **Quick Prototyping**: Test small snippets of code before integrating into larger projects.
2. **Debugging**: Isolate and debug specific parts of your code interactively.
3. **Learning and Experimentation**: Explore new JavaScript features or Node.js modules.

ASSIGNMENT 4:

Node.js REPL Script

Save this script as `customRepl.js`.

```
// Import the readline module for the REPL environment
const readline = require('readline');

// Create a readline interface
const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
    prompt: 'REPL> ' // Custom REPL prompt
});

// Welcome message
console.log("Welcome to the Custom REPL Environment!");
console.log("Type 'help' for a list of available commands or 'exit' to quit.\n");

// Available commands
const commands = {
    help: () => {
        console.log("\nAvailable Commands:");
        console.log("  help       - Display this help message.");
        console.log("  sum(arr)   - Calculate the sum of an array of numbers.");
        console.log("  filter(arr, cond) - Filter an array based on a condition.");
        console.log("  asyncTask  - Simulate an asynchronous operation.");
        console.log("  exit       - Exit the REPL environment.\n");
    },
```

```
    sum: (arr) => {
        const result = arr.reduce((acc, val) => acc + val, 0);
        console.log(`Sum: ${result}`);
    },
    filter: (arr, condition) => {
        const filtered = arr.filter(condition);
        console.log(`Filtered Array: [${filtered}]`);
    },
    asyncTask: () => {
        console.log("Starting an asynchronous task...");
        setTimeout(() => {
            console.log("Asynchronous task completed!");
        }, 2000);
    }
};

// REPL logic
rl.prompt();
rl.on('line', (line) => {
    const input = line.trim();

    try {
        if (input === 'exit') {
            console.log("Goodbye!");
            rl.close();
        } else if (input === 'help') {
            commands.help();
        } else if (input.startsWith('sum(')) {
            // Example: sum([1,2,3,4])
            const arrayString = input.match(/sum\((.*)\)/)[1];
            const array = JSON.parse(arrayString);
            if (Array.isArray(array)) {
                commands.sum(array);
            } else {
                console.log("Error: Argument must be an array.");
            }
        } else if (input.startsWith('filter(')) {
            // Example: filter([1,2,3,4], n => n > 2)
            const match = input.match(/filter\((.*),(.*)\)/);
            if (match) {
                const array = JSON.parse(match[1]);
                const condition = eval(match[2].trim()); // Safe for controlled REPL
                if (Array.isArray(array) && typeof condition === 'function') {
                    commands.filter(array, condition);
                } else {
                    console.log("Error: Invalid input for filter.");
                }
            } else {
                console.log("Error: Invalid syntax for filter.");
            }
        } else if (input === 'asyncTask') {
            commands.asyncTask();
        } else {
            console.log(`Unknown command: ${input}`);
        }
    } catch (error) {
```

```
        console.log(`Error processing input: ${error.message}`);
    }

    rl.prompt();
}).on('close', () => {
    console.log("\nExiting the REPL. Have a great day!");
    process.exit(0);
});
```

Features of the Script

1. **Functions**:
   - `sum(arr)`: Calculates the sum of an array of numbers.
   - `filter(arr, cond)`: Filters an array based on a provided condition (e.g., `n => n > 2`).
2. **Array Methods**:
   - Demonstrates `reduce` for summing and `filter` for filtering.
3. **Asynchronous Operations**:
   - Simulates an asynchronous task using `setTimeout`.
4. **Interactive Commands**:
   - `help`: Displays available commands.
   - `exit`: Exits the REPL environment.

How to Run the Script

1. Save the script in a file named `customRepl.js`.
2. Open a terminal and navigate to the script's directory.
3. Run the script with Node.js:
4. `node customRepl.js`
5. Use the REPL prompt (`REPL>`) to type commands:
   - Example for `sum`:
   - `sum([1,2,3,4])`
   - Example for `filter`:
   - `filter([1,2,3,4], n => n > 2)`
   - Example for asynchronous operation:
   - `asyncTask`
6. Type `exit` to quit the REPL.

ASSIGNMENT 5:

Node.js Program: Calculate Execution Time of Functions

```
// Function 1: Simulates a task with a delay using a loop
function task1() {
    console.log("Executing Task 1...");
    let sum = 0;
    for (let i = 0; i < 1e7; i++) {
        sum += i;
    }
    console.log("Task 1 Result:", sum);
}

// Function 2: Simulates a task using setTimeout (asynchronous)
function task2(callback) {
    console.log("Executing Task 2...");
    setTimeout(() => {
        console.log("Task 2 completed after 2 seconds.");
        callback();
    }, 2000);
}

// Function 3: Processes an array (demonstrates array methods)
function task3() {
    console.log("Executing Task 3...");
    const arr = Array.from({ length: 1e6 }, (_, i) => i);
    const filtered = arr.filter((num) => num % 2 === 0);
    console.log("Task 3 Result:", filtered.length, "even numbers found.");
}

// Measure execution time of Task 1
console.time("Task 1 Execution Time");
task1();
console.timeEnd("Task 1 Execution Time");

// Measure execution time of Task 2
console.time("Task 2 Execution Time");
task2(() => {
    console.timeEnd("Task 2 Execution Time");

    // Measure execution time of Task 3
    console.time("Task 3 Execution Time");
    task3();
    console.timeEnd("Task 3 Execution Time");
});
```

How It Works

1. **`console.time` and `console.timeEnd`:**
   - These methods are used to track the time taken between two points in the code.
   - You pass a label to `console.time` and `console.timeEnd` to identify the measurement.
2. **Tasks:**
   - **Task 1**: Simulates a synchronous computation by summing numbers in a loop.
   - **Task 2**: Simulates an asynchronous task with a 2-second delay using `setTimeout`.
   - **Task 3**: Processes a large array to demonstrate the use of array methods.

3. **Asynchronous Handling:**
   - Task 2 uses a callback to ensure the timing ends only after the asynchronous operation is complete.

Expected Output

When you run the program, the output should look like this:

```
Executing Task 1...
Task 1 Result: 49999995000000
Task 1 Execution Time: 13.456ms

Executing Task 2...
Task 2 completed after 2 seconds.
Task 2 Execution Time: 2002.367ms

Executing Task 3...
Task 3 Result: 500000 even numbers found.
Task 3 Execution Time: 28.734ms
```

How to Run the Program

1. Save the script in a file named `executionTime.js`.
2. Open a terminal and navigate to the file's directory.
3. Run the script using Node.js:
4. `node executionTime.js`

ASSIGNMENT 6:

Node.js Program: Synchronous vs. Asynchronous Functions

```
// Synchronous Function Example
function synchronousFunction() {
    console.log("Synchronous Function Start");
    for (let i = 0; i < 5; i++) {
        console.log(`Processing Synchronous Task ${i + 1}`);
    }
    console.log("Synchronous Function End");
}

// Asynchronous Function Example
function asynchronousFunction() {
    console.log("Asynchronous Function Start");
```

```
    setTimeout(() => {
        console.log("Processing Asynchronous Task after 2 seconds");
    }, 2000);
    console.log("Asynchronous Function End");
}

// Execution Flow
console.log("Program Start");

console.log("\nCalling Synchronous Function:");
synchronousFunction();

console.log("\nCalling Asynchronous Function:");
asynchronousFunction();

console.log("\nProgram End");
```

Explanation of the Code

1. **Synchronous Function:**
   - Executes tasks sequentially, one after the other.
   - Blocks the execution of subsequent lines until the function completes.
   - In the example, `synchronousFunction` processes a loop, blocking other tasks.
2. **Asynchronous Function:**
   - Allows other tasks to execute while waiting for a time-consuming operation to complete.
   - Uses `setTimeout` to simulate a delayed task.
   - In the example, the program continues to execute after starting `asynchronousFunction`, and the callback runs later.
3. **Execution Flow:**
   - The synchronous function executes fully before the asynchronous task's delayed operation begins.

Expected Output

When you run the program, you should see the following output:

```
Program Start

Calling Synchronous Function:
Synchronous Function Start
Processing Synchronous Task 1
Processing Synchronous Task 2
Processing Synchronous Task 3
Processing Synchronous Task 4
Processing Synchronous Task 5
Synchronous Function End

Calling Asynchronous Function:
```

```
Asynchronous Function Start
Asynchronous Function End

Program End
Processing Asynchronous Task after 2 seconds
```

Key Observations

- The synchronous function blocks execution until all its tasks are complete.
- The asynchronous function starts a delayed task and allows the program to proceed, executing the delayed task later.

How to Run the Program

1. Save the script in a file named `syncAsyncExample.js`.
2. Open a terminal and navigate to the script's directory.
3. Run the script with Node.js:
4. `node syncAsyncExample.js`

ASSIGNMENT 7:

---

a) Node.js Program to Describe the Core `path` Module

The `path` module in Node.js is used to handle and transform file paths.

```
// Import the 'path' module
const path = require('path');

// Example: Getting the directory name
const dirName = path.dirname('/user/docs/file.txt');
console.log('Directory Name:', dirName);

// Example: Getting the base name
const baseName = path.basename('/user/docs/file.txt');
console.log('Base Name:', baseName);

// Example: Getting the extension
const extName = path.extname('/user/docs/file.txt');
console.log('Extension Name:', extName);
```

```
// Example: Joining paths
const joinedPath = path.join('/user', 'docs', 'file.txt');
console.log('Joined Path:', joinedPath);

// Example: Resolving paths
const resolvedPath = path.resolve('user', 'docs', 'file.txt');
console.log('Resolved Path:', resolvedPath);

// Example: Parsing a path
const parsedPath = path.parse('/user/docs/file.txt');
console.log('Parsed Path:', parsedPath);

// Example: Formatting a path
const formattedPath = path.format(parsedPath);
console.log('Formatted Path:', formattedPath);
```

Expected Output

```
Directory Name: /user/docs
Base Name: file.txt
Extension Name: .txt
Joined Path: /user/docs/file.txt
Resolved Path: /<current_working_directory>/user/docs/file.txt
Parsed Path: { root: '/', dir: '/user/docs', base: 'file.txt', ext: '.txt', name:
'file' }
Formatted Path: /user/docs/file.txt
```

---

b) Node.js Program to Describe Core Modules `path` and `fs`

The `fs` (File System) module is used to interact with the file system.

```
// Import the 'path' and 'fs' modules
const path = require('path');
const fs = require('fs');

// Example: Using the 'path' module
const filePath = path.join(__dirname, 'example.txt');
console.log('File Path:', filePath);

// Example: Writing to a file using 'fs'
fs.writeFile(filePath, 'This is an example file.', (err) => {
    if (err) {
        return console.error('Error writing to file:', err);
    }
    console.log('File written successfully.');

    // Reading the file using 'fs'
    fs.readFile(filePath, 'utf8', (err, data) => {
        if (err) {
            return console.error('Error reading the file:', err);
        }
```

```
        console.log('File Content:', data);

        // Deleting the file using 'fs'
        fs.unlink(filePath, (err) => {
            if (err) {
                return console.error('Error deleting the file:', err);
            }
            console.log('File deleted successfully.');
        });
    });
});
```

Expected Output

```
File Path: /<current_working_directory>/example.txt
File written successfully.
File Content: This is an example file.
File deleted successfully.
```

Explanation

1. **`path` Module**:
   - Handles file and directory paths effectively (e.g., joining, resolving, parsing).
2. **`fs` Module**:
   - **`writeFile`**: Writes content to a file.
   - **`readFile`**: Reads content from a file.
   - **`unlink`**: Deletes a file.

How to Run the Programs

1. Save the first script as `pathModule.js` and the second as `pathFsModules.js`.
2. Run them using Node.js:
3. `node pathModule.js`
4. `node pathFsModules.js`

ASSIGNMENT 8:

Node.js Program: Demonstrating Buffers

```
// Creating Buffers with Buffer.from() and Buffer.alloc()

// Buffer.from() creates a buffer from a string
const bufferFrom = Buffer.from('Hello, Buffer!');
```

```
console.log('Buffer created with Buffer.from():', bufferFrom);
console.log('String from buffer:', bufferFrom.toString());

// Buffer.alloc() creates a buffer of a specific size and fills it with zeros
const bufferAlloc = Buffer.alloc(15); // Allocates 15 bytes
console.log('Buffer created with Buffer.alloc():', bufferAlloc);

// Writing Data to Buffers
bufferAlloc.write('Node.js Buffers', 0, 'utf8');
console.log('Buffer after write():', bufferAlloc);
console.log('String from buffer after write():', bufferAlloc.toString());

// Slicing Buffers
const slicedBuffer = bufferFrom.slice(7, 13); // Extracts "Buffer" from "Hello,
Buffer!"
console.log('Sliced Buffer:', slicedBuffer);
console.log('String from sliced buffer:', slicedBuffer.toString());

// Modifying Sliced Buffer and Original Buffer Behavior
slicedBuffer.write('Bytes');
console.log('Modified Sliced Buffer:', slicedBuffer.toString());
console.log('Original Buffer after slicing:', bufferFrom.toString()); // Observe the
effect on the original buffer
```

Explanation

1. **Creating Buffers:**
   - `Buffer.from()` creates a buffer initialized with specific data (e.g., a string or array).
   - `Buffer.alloc()` creates a zero-filled buffer of the specified size.
2. **Writing Data:**
   - `write()` writes data into the buffer at a specified offset.
   - In this example, `"Node.js Buffers"` is written to the `bufferAlloc` instance.
3. **Slicing Buffers:**
   - `slice()` creates a new buffer sharing the same memory as the original buffer. Changes to the sliced buffer affect the original buffer.

Expected Output

```
Buffer created with Buffer.from(): <Buffer 48 65 6c 6c 6f 2c 20 42 75 66 66 65 72 21>
String from buffer: Hello, Buffer!
Buffer created with Buffer.alloc(): <Buffer 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00>
Buffer after write(): <Buffer 4e 6f 64 65 2e 6a 73 20 42 75 66 66 65 72 73>
String from buffer after write(): Node.js Buffers
Sliced Buffer: <Buffer 42 75 66 66 65 72>
String from sliced buffer: Buffer
Modified Sliced Buffer: Bytes
Original Buffer after slicing: Hello, Bytes!
```

Key Points

1. **Buffers in Node.js**:
   - Buffers are used to handle binary data.
   - Common in file systems, streams, and network operations.
2. **Shared Memory in `slice()`**:
   - Changes to the sliced buffer can affect the original buffer because they share memory.
3. **Memory Allocation**:
   - Use `Buffer.alloc()` for predictable, zero-filled buffers.
   - Avoid `Buffer.allocUnsafe()` unless performance optimization is necessary.

How to Run

1. Save the script as `bufferExample.js`.
2. Run the script using Node.js:
3. `node bufferExample.js`

ASSIGNMENT 9:

---

a) Node.js Program to Describe Local Modules with Two Functions

In Node.js, a **local module** is a custom module that you create for use within your application. It can contain various functions, variables, or any other logic you want to use across multiple files.

Here's a Node.js program that demonstrates a local module with two functions.

Step 1: Create the Local Module (mathFunctions.js)

This module will contain two functions: one for addition and one for multiplication.

```
// mathFunctions.js (Local Module)

// Function to add two numbers
function add(a, b) {
    return a + b;
}

// Function to multiply two numbers
function multiply(a, b) {
```

```
    return a * b;
}

// Exporting functions to make them available in other files
module.exports = {
    add,
    multiply
};
```

Step 2: Use the Local Module (app.js)

Now, you can use the `mathFunctions.js` module in your main `app.js` file by requiring it.

```
// app.js (Main Application)

// Importing the local module
const math = require('./mathFunctions');

// Using the functions from the local module
const sum = math.add(5, 3);
const product = math.multiply(5, 3);

console.log(`Sum: ${sum}`);          // Sum: 8
console.log(`Product: ${product}`); // Product: 15
```

Expected Output

```
Sum: 8
Product: 15
```

---

b) Node.js Program to Describe Local Modules with an Object

In this example, we'll create a local module that exports an object, which can contain various properties and methods.

Step 1: Create the Local Module (person.js)

This module will define a person object with some properties and methods.

```
// person.js (Local Module)

// Defining an object with properties and methods
const person = {
    firstName: 'John',
    lastName: 'Doe',
    age: 30,
```

```
    // Method to display full name
    getFullName: function() {
        return `${this.firstName} ${this.lastName}`;
    },

    // Method to increase age by one
    celebrateBirthday: function() {
        this.age++;
    }
};

// Exporting the person object to make it available in other files
module.exports = person;
```

Step 2: Use the Local Module (app.js)

Now, let's require and use the `person.js` module in the main `app.js` file.

```
// app.js (Main Application)

// Importing the local module
const person = require('./person');

// Displaying the person object properties and using methods
console.log(`Full Name: ${person.getFullName()}`);   // Full Name: John Doe
console.log(`Age: ${person.age}`);                   // Age: 30

// Calling method to celebrate birthday
person.celebrateBirthday();

console.log(`New Age after Birthday: ${person.age}`); // New Age after Birthday: 31
```

Expected Output

```
Full Name: John Doe
Age: 30
New Age after Birthday: 31
```

Explanation

1. **Local Modules**:
   - You create local modules by writing JavaScript code in separate files (e.g., `mathFunctions.js`, `person.js`).
   - You export the functions or objects you want to share using `module.exports`.
2. **Functions Exported in a Module**:
   - In the first example, the module exports two functions (`add` and `multiply`), which are used in the main application.

3. **Object Exported in a Module**:
   - In the second example, an object `person` is created with properties (`firstName`, `lastName`, `age`) and methods (`getFullName`, `celebrateBirthday`).
   - This object is exported and used in the main application.

How to Run

1. Save the first script as `mathFunctions.js` and the second one as `app.js`.
2. Save the second script as `person.js` and `app.js`.
3. Run the scripts using Node.js:
4. `node app.js`

ASSIGNMENT 10:

Creating a `package.json` File and Setting Up with `dotenv`

In Node.js, `package.json` is a file that contains metadata about your project, such as its name, version, dependencies, and scripts. It's also used to manage external packages or modules.

Step 1: Create `package.json` File

To create a `package.json` file, follow these steps:

1. Open a terminal/command prompt.
2. Navigate to your Node.js project folder (or create a new folder if it doesn't exist).
3. Run the following command to generate a `package.json` file:
4. `npm init -y`

This will generate a basic `package.json` file with default values.

Step 2: Install `dotenv` Package

`dotenv` is an external package that loads environment variables from a `.env` file into `process.env`. This is useful for managing sensitive information, like API keys, without hardcoding them into your code.

To install `dotenv`, run the following command:

```
npm install dotenv
```

This will add `dotenv` to your project's dependencies in the `package.json` file.

Step 3: Create `.env` File

Create a `.env` file in the root of your project folder to store environment variables. Example:

```
# .env
DB_HOST=localhost
DB_USER=root
DB_PASS=password123
```

This `.env` file will contain the environment variables that you can later access in your code.

Step 4: Use `dotenv` in Your Code

Now, you can use the `dotenv` package in your code to load the environment variables from the `.env` file.

```
// app.js

// Load environment variables from the .env file
require('dotenv').config();

// Access environment variables
const dbHost = process.env.DB_HOST;
const dbUser = process.env.DB_USER;
const dbPass = process.env.DB_PASS;

console.log(`Database Host: ${dbHost}`);
console.log(`Database User: ${dbUser}`);
console.log(`Database Password: ${dbPass}`);
```

Step 5: Final `package.json` Example

After installing `dotenv`, your `package.json` file might look like this:

```
{
  "name": "my-node-project",
  "version": "1.0.0",
  "description": "",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "dotenv": "^16.0.0"
```

```
  },
  "author": "",
  "license": "ISC"
}
```

Expected Output

When you run the program using `node app.js`, you should see:

```
Database Host: localhost
Database User: root
Database Password: password123
```

How to Run

1. Save the JavaScript code as `app.js`.
2. Create a `.env` file in the root of your project.
3. Run the script:
4. `node app.js`

ASSIGNMENT 11:

---

a) Node.js Program to Create a Server Using the `http` Core Module

This basic program demonstrates how to create a server using the `http` core module in Node.js.

Step 1: Create a Simple HTTP Server (server.js)

```
// Import the http core module
const http = require('http');

// Define the server
const server = http.createServer((req, res) => {
    // Set the response header
    res.setHeader('Content-Type', 'text/plain');

    // Send a response to the client
    res.end('Hello, World!');
});

// Define the port and start the server
const port = 3000;
server.listen(port, () => {
    console.log(`Server running at http://localhost:${port}/`);
});
```

Explanation

1. **http.createServer()** creates an HTTP server.
2. The callback function inside `createServer` is called for every request. It receives two arguments:
   - `req` (request): Contains information about the incoming request.
   - `res` (response): Used to send the response to the client.
3. The `res.setHeader()` method is used to set the HTTP response headers.
4. The `res.end()` method sends the response to the client.
5. The server listens on port `3000`.

How to Run the Program

1. Save the code as `server.js`.
2. Open a terminal and run the script:
3. `node server.js`
4. Open a browser and navigate to `http://localhost:3000/`. You should see `Hello, World!`.

---

b) Extend the Server to Handle Routes and Process Different HTTP Methods (GET, POST, PUT, DELETE)

We will extend the server to handle different routes and HTTP methods like `GET`, `POST`, `PUT`, and `DELETE`.

Step 2: Extend the Server to Handle Routes and HTTP Methods

```
// Import the http core module
const http = require('http');

// Define the server
const server = http.createServer((req, res) => {
    // Set the response header
    res.setHeader('Content-Type', 'application/json');

    // Handle different HTTP methods and routes
    if (req.method === 'GET' && req.url === '/get-data') {
        // Handle GET request for /get-data
        res.statusCode = 200;
        res.end(JSON.stringify({ message: 'GET request received' }));
    }
    else if (req.method === 'POST' && req.url === '/post-data') {
        // Handle POST request for /post-data
        let body = '';
        req.on('data', chunk => {
```

```
            body += chunk;
        });
        req.on('end', () => {
            res.statusCode = 201;
            res.end(JSON.stringify({ message: 'POST request received', data:
JSON.parse(body) }));
        });
    }
    else if (req.method === 'PUT' && req.url === '/put-data') {
        // Handle PUT request for /put-data
        let body = '';
        req.on('data', chunk => {
            body += chunk;
        });
        req.on('end', () => {
            res.statusCode = 200;
            res.end(JSON.stringify({ message: 'PUT request received', data:
JSON.parse(body) }));
        });
    }
    else if (req.method === 'DELETE' && req.url === '/delete-data') {
        // Handle DELETE request for /delete-data
        res.statusCode = 200;
        res.end(JSON.stringify({ message: 'DELETE request received' }));
    }
    else {
        // Handle all other requests
        res.statusCode = 404;
        res.end(JSON.stringify({ message: 'Not Found' }));
    }
});

// Define the port and start the server
const port = 3000;
server.listen(port, () => {
    console.log(`Server running at http://localhost:${port}/`);
});
```

Explanation

1. **GET Method** (`/get-data`):
   - The server responds with a JSON message indicating that a GET request was received.
2. **POST Method** (`/post-data`):
   - The server listens for incoming data in the request body and responds with the received data.
   - `req.on('data')` collects the incoming data, and `req.on('end')` processes it after all data is received.
3. **PUT Method** (`/put-data`):
   - Similar to the POST method, the server listens for data, updates it, and responds with the updated content.
4. **DELETE Method** (`/delete-data`):
   - The server sends a response indicating that the DELETE request was received.

5. **404 Handling**:
   - If the request doesn't match any of the above conditions, a 404 Not Found response is sent.

How to Test the Program

1. Save the code as `server.js`.
2. Open a terminal and run the script:
3. `node server.js`
4. You can test each route using a tool like **Postman** or **curl**.

Test GET Request

```
curl http://localhost:3000/get-data
```

Expected Output:

```
{"message": "GET request received"}
```

Test POST Request

```
curl -X POST http://localhost:3000/post-data -d '{"name":"John"}' -H "Content-Type: application/json"
```

Expected Output:

```
{"message": "POST request received", "data": {"name":"John"}}
```

Test PUT Request

```
curl -X PUT http://localhost:3000/put-data -d '{"name":"Doe"}' -H "Content-Type: application/json"
```

Expected Output:

```
{"message": "PUT request received", "data": {"name":"Doe"}}
```

Test DELETE Request

```
curl -X DELETE http://localhost:3000/delete-data
```

Expected Output:

```
{"message": "DELETE request received"}
```

Test Unknown Route

```
curl http://localhost:3000/unknown-route
```

Expected Output:

```
{"message": "Not Found"}
```

Expected Output on Console

```
Server running at http://localhost:3000/
```

This extended example demonstrates how to create a basic server in Node.js that handles various HTTP methods and routes, enabling you to perform CRUD-like operations.

ASSIGNMENT 12:

Here's a Node.js program that demonstrates CRUD operations (Create, Read, Update, Delete) on a file using the `fs` (File System) core module:

Node.js Program: CRUD Operations on a File

This program will perform the following operations on a file named `example.txt`:

1. **Create** - Create a new file or overwrite an existing one.
2. **Read** - Read the content of the file.
3. **Update** - Append new data to the file.
4. **Delete** - Delete the file.

Step 1: Create the Node.js Program (crud.js)

```
const fs = require('fs');
const filePath = 'example.txt';

// Create operation: Write to a file (Create or Overwrite)
function createFile() {
    const content = 'Hello, this is a Node.js file!';
    fs.writeFile(filePath, content, 'utf8', (err) => {
        if (err) {
            console.log('Error creating file:', err);
        } else {
            console.log('File created and written successfully!');
        }
    });
}

// Read operation: Read from the file
function readFile() {
    fs.readFile(filePath, 'utf8', (err, data) => {
        if (err) {
            console.log('Error reading file:', err);
        } else {
            console.log('File content:', data);
```

```
        }
    });
}

// Update operation: Append to the file
function updateFile() {
    const newContent = '\nThis is the updated content!';
    fs.appendFile(filePath, newContent, 'utf8', (err) => {
        if (err) {
            console.log('Error updating file:', err);
        } else {
            console.log('File updated successfully!');
        }
    });
}

// Delete operation: Delete the file
function deleteFile() {
    fs.unlink(filePath, (err) => {
        if (err) {
            console.log('Error deleting file:', err);
        } else {
            console.log('File deleted successfully!');
        }
    });
}

// Run CRUD operations
createFile();    // Creates or overwrites the file
setTimeout(readFile, 1000);   // Read file content after 1 second
setTimeout(updateFile, 2000); // Update file content after 2 seconds
setTimeout(deleteFile, 3000); // Delete file after 3 seconds
```

Explanation

1. **Create** (`writeFile`):
   - `fs.writeFile(filePath, content, 'utf8', callback)` creates a new file or overwrites an existing file with the specified content.
2. **Read** (`readFile`):
   - `fs.readFile(filePath, 'utf8', callback)` reads the content of the file. If the file doesn't exist, it throws an error.
3. **Update** (`appendFile`):
   - `fs.appendFile(filePath, newContent, 'utf8', callback)` appends data to the file. If the file does not exist, it is created.
4. **Delete** (`unlink`):
   - `fs.unlink(filePath, callback)` deletes the file.

How to Run the Program

1. Save the code as `crud.js`.
2. Open a terminal and run the script:
3. `node crud.js`
4. Observe the following operations:
    - A file named `example.txt` will be created and written to.
    - After 1 second, the content of the file will be displayed in the console.
    - After 2 seconds, additional content will be appended to the file.
    - After 3 seconds, the file will be deleted.

Expected Output

```
File created and written successfully!
File content: Hello, this is a Node.js file!
File updated successfully!
File deleted successfully!
```

Notes

- If the file already exists, it will be overwritten during the **Create** operation.
- You can adjust the `setTimeout` durations based on your needs for observing each operation's sequence.

ASSIGNMENT 13:

---

a) Create a Node.js Project Named "Calculator"

Below is the implementation for your request with small code examples. The project will have the following features:

1. A home page (`/`) displaying a welcome message and a link to the calculator page.
2. A calculator page (`/calculator`) with a form to input two numbers and calculate their sum.
3. A result page (`/calculate-result`) parsing user input, calculating the sum using a separate addition module, and displaying the result.

Step 1: Project Structure

```
calculator/
│
├── app.js            # Main server file
├── add.js            # Addition module
```

```
└── package.json     # Project metadata
```

Step 2: Create the `package.json` File

Run the following command in your terminal to initialize the project:

```
npm init -y
```

Install the required packages:

```
npm install express
```

Step 3: Create the Addition Module (`add.js`)

This module contains a simple function to add two numbers.

```
// add.js
function add(a, b) {
    return a + b;
}

module.exports = add;
```

Step 4: Create the Main Server File (`app.js`)

```
const express = require('express');
const bodyParser = require('body-parser');
const add = require('./add'); // Import the addition module

const app = express();
const port = 3000;

// Middleware to parse form data
app.use(bodyParser.urlencoded({ extended: true }));

// Home route
app.get('/', (req, res) => {
    res.send(`
        <h1>Welcome to the Calculator App</h1>
        <p><a href="/calculator">Go to Calculator</a></p>
    `);
});

// Calculator route
app.get('/calculator', (req, res) => {
    res.send(`
        <h1>Calculator</h1>
        <form action="/calculate-result" method="POST">
```

```
            <input type="number" name="num1" placeholder="Enter first number"
required>
            <input type="number" name="num2" placeholder="Enter second number"
required>
            <button type="submit">Calculate Sum</button>
        </form>
    `);
});

// Result route to handle calculation
app.post('/calculate-result', (req, res) => {
    const num1 = parseFloat(req.body.num1);
    const num2 = parseFloat(req.body.num2);
    const result = add(num1, num2); // Call the add function from add.js
    res.send(`
        <h1>Result</h1>
        <p>The sum of ${num1} and ${num2} is ${result}</p>
        <p><a href="/calculator">Go back to Calculator</a></p>
    `);
});

// Start the server
app.listen(port, () => {
    console.log(`Server is running at http://localhost:${port}`);
});
```

Step 5: Run the Server

1. Open a terminal and navigate to your project directory.
2. Run the server with the following command:

```
node app.js
```

3. Open a browser and navigate to `http://localhost:3000/`.

Explanation

1. **Home Page (`/`)**:
   - Displays a simple welcome message and a link to the `/calculator` page.
2. **Calculator Page (`/calculator`)**:
   - Displays a form where users can input two numbers.
3. **Result Page (`/calculate-result`)**:
   - Handles the form submission via a POST request.
   - Parses the input values and calculates their sum using the `add` function from the `add.js` module.
   - Displays the result of the calculation.

Expected Output

- **Home Page (`/`)**:
- `<h1>Welcome to the Calculator App</h1>`
- `<p><a href="/calculator">Go to Calculator</a></p>`
- **Calculator Page (`/calculator`)**:
- `<h1>Calculator</h1>`
- `<form action="/calculate-result" method="POST">`
- `    <input type="number" name="num1" placeholder="Enter first number" required>`
- `    <input type="number" name="num2" placeholder="Enter second number" required>`
- `    <button type="submit">Calculate Sum</button>`
- `</form>`
- **Result Page (`/calculate-result`)** (Example with input `5` and `3`):
- `<h1>Result</h1>`
- `<p>The sum of 5 and 3 is 8</p>`
- `<p><a href="/calculator">Go back to Calculator</a></p>`

Conclusion

This is a simple Node.js application with Express that:

- Serves a home page with a link to the calculator page.
- Provides a form to calculate the sum of two numbers.
- Uses a separate addition module to calculate the result and display it.

ASSIGNMENT 14:

Node.js Project Using Express

In this example, we will create a simple Express app that:

1. Responds with both **HTML** and **JSON** at different routes.
2. Serves different HTML files at different routes.

Step 1: Project Structure

```
express-project/
│
├── app.js           # Main server file
├── views/
│   ├── home.html    # HTML file for the home route
│   ├── about.html   # HTML file for the about route
└── package.json     # Project metadata
```

Step 2: Initialize the Project

Run the following command in your terminal to initialize the project:

```
npm init -y
```

Install the required packages:

```
npm install express
```

Step 3: Create the Main Server File (`app.js`)

```
const express = require('express');
const path = require('path');

const app = express();
const port = 3000;

// Set the view engine to serve HTML files from the 'views' directory
app.set('view engine', 'html');
app.set('views', path.join(__dirname, 'views'));

// Middleware to parse JSON
app.use(express.json());

// Home route (HTML response)
app.get('/', (req, res) => {
    res.sendFile(path.join(__dirname, 'views', 'home.html'));
});

// About route (HTML response)
app.get('/about', (req, res) => {
    res.sendFile(path.join(__dirname, 'views', 'about.html'));
});

// JSON response route
app.get('/json', (req, res) => {
    const data = {
        message: "Welcome to the API!",
        status: "success"
```

```
    };
    res.json(data);
});

// Start the server
app.listen(port, () => {
    console.log(`Server is running at http://localhost:${port}`);
});
```

Step 4: Create HTML Files in the `views` Folder

home.html

```
<!-- views/home.html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Home Page</title>
</head>
<body>
    <h1>Welcome to the Home Page!</h1>
    <p><a href="/about">Go to About Page</a></p>
    <p><a href="/json">Go to JSON Response</a></p>
</body>
</html>
```

about.html

```
<!-- views/about.html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>About Page</title>
</head>
<body>
    <h1>About Us</h1>
    <p>This is the About Page.</p>
    <p><a href="/">Go back to Home Page</a></p>
    <p><a href="/json">Go to JSON Response</a></p>
</body>
</html>
```

Step 5: Run the Server

1. Open a terminal and navigate to your project directory.
2. Run the server with the following command:

```
node app.js
```

3. Open a browser and navigate to `http://localhost:3000/`.

Expected Output

- **Home Route (`/`)**:
  - Displays the **home.html** file with a welcome message and links to the about page and JSON response.
- **About Route (`/about`)**:
  - Displays the **about.html** file with information about the page and links to the home page and JSON response.
- **JSON Route (`/json`)**:
  - Displays the following JSON response:
- {
- "message": "Welcome to the API!",
- "status": "success"
- }

Conclusion

This simple Node.js project demonstrates how to:

1. Respond with **HTML** files at different routes (`/` and `/about`).
2. Respond with **JSON** at the `/json` route.

ASSIGNMENT 15:

Extended Express Project

In this extended project, we'll:

1. **Serve different HTML files** at different routes and **perform a calculation** (e.g., adding two numbers from a user form).
2. **Implement routing with dynamic parameters** in the URL.
3. Use **application-level middleware** for tasks like logging or handling errors.

Project Structure

```
express-project/
│
├── app.js           # Main server file
├── views/
│   ├── home.html    # HTML file for the home route
│   ├── calculator.html  # HTML file with a form for the calculator
│   └── result.html  # HTML file to display the result of the calculation
└── package.json     # Project metadata
```

Step 1: Initialize the Project

Make sure you have initialized the project and installed the required packages (`express`):

```
npm init -y
npm install express
```

Step 2: Create the Main Server File (`app.js`)

```
const express = require('express');
const path = require('path');
const bodyParser = require('body-parser');

const app = express();
const port = 3000;

// Middleware for logging requests
app.use((req, res, next) => {
    console.log(`${req.method} request to ${req.url}`);
    next();
});

// Middleware for parsing form data (application/x-www-form-urlencoded)
app.use(bodyParser.urlencoded({ extended: true }));

// Set the view engine to serve HTML files from the 'views' directory
app.set('view engine', 'html');
app.set('views', path.join(__dirname, 'views'));

// Home route (serves home.html)
app.get('/', (req, res) => {
    res.sendFile(path.join(__dirname, 'views', 'home.html'));
});

// Calculator route (serves calculator.html)
app.get('/calculator', (req, res) => {
    res.sendFile(path.join(__dirname, 'views', 'calculator.html'));
});

// Result route to perform calculation (serves result.html)
```

```
app.post('/calculate', (req, res) => {
    const num1 = parseFloat(req.body.num1);
    const num2 = parseFloat(req.body.num2);
    const result = num1 + num2;

    res.sendFile(path.join(__dirname, 'views', 'result.html'), {
        headers: {
            'Content-Type': 'text/html'
        },
        statusCode: 200
    });
});

// Dynamic route with parameters (example: /greet/:name)
app.get('/greet/:name', (req, res) => {
    const name = req.params.name;
    res.send(`<h1>Hello, ${name}!</h1>`);
});

// Error handling middleware
app.use((req, res) => {
    res.status(404).send('<h1>404: Page Not Found</h1>');
});

// Start the server
app.listen(port, () => {
    console.log(`Server is running at http://localhost:${port}`);
});
```

Step 3: Create HTML Files in the `views` Folder

home.html

```
<!-- views/home.html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Home Page</title>
</head>
<body>
    <h1>Welcome to the Calculator App</h1>
    <p><a href="/calculator">Go to Calculator</a></p>
</body>
</html>
```

calculator.html

```
<!-- views/calculator.html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Calculator</title>
```

```
</head>
<body>
    <h1>Calculator</h1>
    <form action="/calculate" method="POST">
        <input type="number" name="num1" placeholder="Enter first number" required>
        <input type="number" name="num2" placeholder="Enter second number" required>
        <button type="submit">Calculate</button>
    </form>
</body>
</html>
```

result.html

```
<!-- views/result.html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Calculation Result</title>
</head>
<body>
    <h1>Calculation Result</h1>
    <p>The sum is: {{result}}</p>
    <p><a href="/calculator">Go back to Calculator</a></p>
</body>
</html>
```

Step 4: Run the Server

1. Open a terminal and navigate to your project directory.
2. Run the server with the following command:

```
node app.js
```

3. Open a browser and navigate to `http://localhost:3000/`.

Step 5: Test the Routes

1. **Home Route (`/`)**:
   - Displays the **home.html** page with a welcome message and a link to the calculator page.
2. **Calculator Route (`/calculator`)**:
   - Displays the **calculator.html** page with a form for the user to input two numbers.
3. **Result Route (`/calculate`)**:
   - Submits the form data via a `POST` request.
   - The server calculates the sum and sends back the **result.html** page with the sum.
4. **Dynamic Route (`/greet/:name`)**:
   - You can test this route by visiting `http://localhost:3000/greet/John` to see a personalized greeting.

5. **Error Handling**:
   - If you visit a non-existent route (e.g., `/nonexistent`), the **404 error handler** will respond with a 404 page.

Conclusion

In this extended Node.js project, we:

- Served different HTML files at different routes.
- Implemented a basic calculator that adds two numbers submitted via a form.
- Used dynamic routes with URL parameters (e.g., `/greet/:name`).
- Added application-level middleware for logging and error handling.

6. Precautions and/or Troubleshooting

Ensure proper installation of dependencies, correct error handling, static file management, body parsing, input validation, route definition, and middleware order to avoid issues in your Node.js Express project.

7. Observations

Node.js Express project demonstrates efficient asynchronous handling, scalability, middleware use, flexible route management, dependency management, and the importance of input validation, error handling, and performance optimization.

8. Calculations & Analysis

The Node.js Express project demonstrates high performance, low memory usage, scalability, and low error rates, with optimized response times and throughput, while emphasizing the importance of security and error handling.

9. Result & Interpretation

The Node.js Express project successfully delivers fast, scalable, and efficient handling of requests, with secure data processing and optimized performance, ensuring a reliable user experience.

10. Follow-up Questions

1. How does the event-driven architecture of Node.js contribute to its performance?
2. What is the purpose of middleware in Express, and how would you create custom middleware?
3. Explain how routing works in Express and how dynamic URL parameters can be handled.
4. How do you handle errors in Express applications, and why is error handling crucial for production environments?
5. What are the advantages of using `express.static()` to serve static files, and how do you ensure correct file paths?
6. Describe how you would perform input validation and sanitization to prevent common security vulnerabilities like SQL injection.
7. What are the differences between synchronous and asynchronous functions in Node.js, and how do they impact performance?
8. How can you scale a Node.js application to handle a large number of concurrent users?
9. What strategies can be used to improve the response time of an Express application that performs complex database queries?
10. In what scenarios would you choose to use Express over other frameworks, and what are its limitations?

11. Extension and Follow-up Activities (if applicable):

Not applicable

12. Assessments

As per the assessment and evaluation policy of University

13. Suggested readings :

1) **"Node.js Design Patterns" by Mario Casciaro** – A comprehensive guide to understanding the internal workings and design patterns in Node.js applications.
2) **"Express in Action" by Evan Hahn** – A practical guide for learning how to build web applications using Express.js.
3) **"Learning Node.js Development" by Andrew Mead** – A beginner-friendly resource for building and deploying real-world applications with Node.js.
4) **"You Don't Know JS" series by Kyle Simpson** – A deep dive into JavaScript fundamentals, which is essential when working with Node.js.
5) **"JavaScript: The Good Parts" by Douglas Crockford** – A book that focuses on the best parts of JavaScript, critical for mastering Node.js development.

14. Assignment copy format:

1. Write in A4 page; No channel file will be accepted.
2. Front page and Index page format will be provided by department
3. Following points must be included while writing assignment copy
    a) Problem definition
    b) Algorithm
    c) Program in corresponding language
    d) Output